Hand Gesture Recognition for Interactive Puzzle Game Control

A Comparative Analysis of Heuristic, Classical ML, and Neural Network Approaches

Dinh Thanh Hieu (22BA13132) Nguyen Thai Nhat Anh (23BI14024) Le Quoc Anh (23BI14025)

Instructor: Prof. Nguyen Duc Dung
Vietnam Academy of Science and Technology
University of Science and Technology of Hanoi

February 2026

Motivation: Toward Natural Human-Computer Interaction

- **The Core Problem:** Traditional input devices (keyboard, mouse) impose a barrier between user intent and system response. The physical necessity of moving hands off a task to utilize a peripheral disrupts immersive workflows.
- **The Solution:** Hand gestures offer an intuitive, hardware-free alternative by tracking spatial intent directly through a standard webcam.
- **The Engineering Challenge:** Hand morphology varies wildly across users (size, joint flexibility, skin tone). Illumination and viewing angles add massive variance.
- **Research Question:** How can we build a real-time gesture sequence pipeline that generalizes reliably to unseen users without relying on computational-heavy Deep Neural Networks solely?

Objectives: Three Methods, One Benchmark

- **Objective 1:** Implement and compare three gesture classification methods spanning the complexity spectrum:
  - Rule-based heuristic (no training data)
  - Random Forest (engineered features)
  - Multi-Layer Perceptron (raw landmark coordinates)
- **Objective 2:** Evaluate all methods using person-aware cross-validation on a large-scale public dataset.
- **Objective 3:** Deploy the highest-performing model in a real-time browser-based application.

Heuristic (Rules) → Random Forest (Trees) → MLP (Network)

Increasing model complexity

Five Gesture Classes, Five Game Actions

Gesture	Description	Game Action
Open Hand	Spread fingers, palm forward	Release puzzle piece
Fist	Closed hand	Rotate puzzle piece
Pinch	Thumb and index finger contact	Grab / release piece
Frame	Two hands forming a rectangle	Capture puzzle image
None	No defined gesture	No action

End-to-End Pipeline: Webcam to Game Control

1. **Input Acquisition:** Video frame captured from webcam
2. **Landmark Extraction:** Google MediaPipe HandLandmarker produces 21 3D landmarks
3. **Normalization:** Position- and scale-invariant preprocessing
4. **Classification:** Gesture prediction via one of three evaluated methods
5. **Application:** Predicted gesture triggers corresponding game action

$$\text{Video Frame} \xrightarrow{\text{MediaPipe}} \mathbf{L} \in \mathbb{R}^{21\times 3} \xrightarrow{\text{Classifier}} y \in \{1, \ldots, 5\}$$

Training Data: The HaGRID v2 Dataset Validation

Source Dataset Context:

- Subsampled from **HaGRID v2** containing over 1 million frames and 37,583 subjects.
- Crucial feature: It provides pre-annotated **2D coordinates** (x, y) but depth $z = 0.0$, forming an explicit limitation for algorithms relying on 3D geometry.
- **Extraction:** 9,231 stratified samples extracted across 3,607 unique individual identities.

Class	Train	Val	Total
open_hand	1,000	1,000	2,000
fist	1,000	1,000	2,000
pinch	1,000	1,000	2,000
frame	1,000	1,000	2,000
none	1,000	231	1,231
Total	**5,000**	**4,231**	**9,231**

Feature Extraction: Landmark Normalization Invariance

Problem: Raw coordinates break classifiers the moment a user moves away from the camera or shifts their hand.

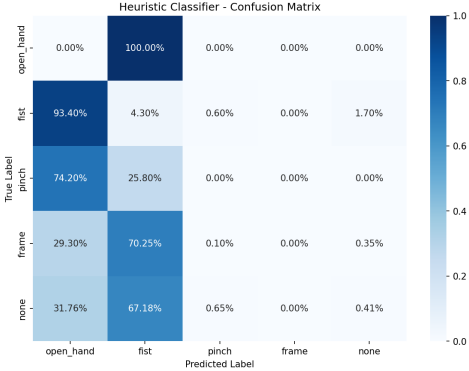
Solution: Standardize orientation and scale using a vector pipeline:

- **Translation Invariance:** We shift the mathematical space so the Wrist (Index 0) acts as coordinates $(0, 0)$. This makes model blind to *where* the hand is in the video.
- **Scale Invariance:** We divide coordinates by the Euclidean distance to the Middle Finger joint (Index 9). This makes the model blind to *hand dimension size*.
- **Result:** A pure **60-dimensional structural vector** per frame (20 joints $\times$ 3 axes).

$$\hat{\mathbf{l}}_i = \frac{\mathbf{l}_i - \mathbf{l}_0}{\|\mathbf{l}_9 - \mathbf{l}_0\|_2}, \quad \text{for } i = 1, \ldots, 20$$

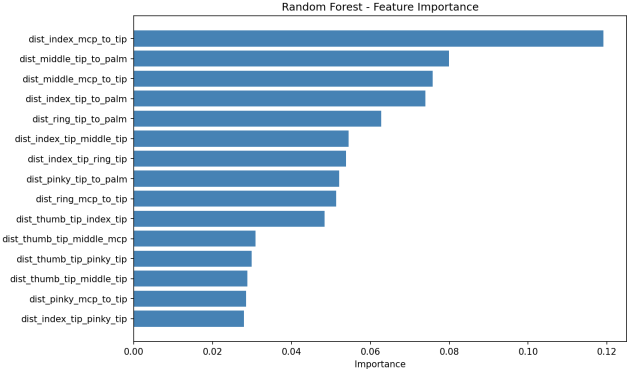
Method 1: Rule-Based Heuristic Classification

- **Approach:** ∼20 manually calibrated thresholds on finger curl angles and inter-landmark distances.
- **Result:** **0.99% accuracy**—below random baseline of 20%!
- **Reason why this is crucial to present:** It validates our data limitations. The HaGRID dataset is 2D ($z = 0$); attempting 3D geometry causes vector calculations to degenerate to random noise. If we had 3D ground truth, Heuristic models would likely perform exceptionally fast.



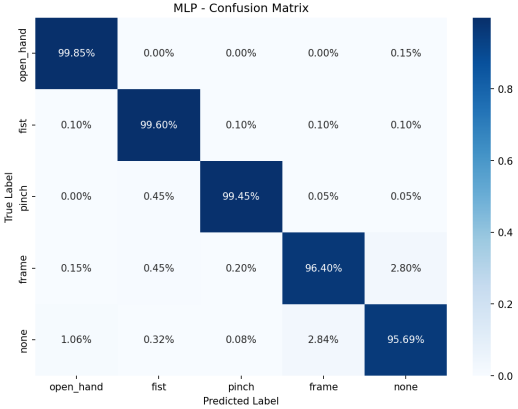
Method 2: Random Forest with Engineered Features

- **Configuration:** 100 Trees (`max_depth=15`)
- **Input:** Instead of 60 raw coords, we engineer a condensed 24-dim vector of pairwise Euclidean distances.
- **Result: 94.13% accuracy** (13.2 ms latency).
- **Strength:** Highly interpretable, we can see exactly which distances contribute to choices.



Method 3: Multi-Layer Perceptron on Raw Landmarks

- **Architecture:** A 2-Hidden-Layer Network ($60 \rightarrow 128 \rightarrow 64 \rightarrow 5$) with Dropout regularization.
- **Size:** Barely 14,789 parameters (227 KB footprint).
- **Result:** Peak **98.41% accuracy**, 43.7 ms inference latency. No feature engineering needed.



Three Methods: Head-to-Head Comparison

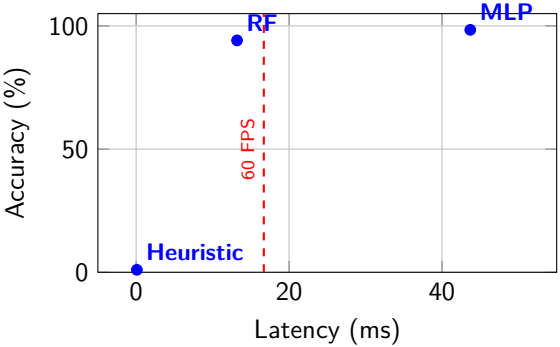
Criterion	**Heuristic**	**Random Forest**	**MLP**
Input	Angles + dist.	24 pairwise dist.	60 raw coords
Feature Eng.	Manual	Semi-automatic	None (end-to-end)
Training	No	Yes	Yes
Parameters	$\sim$20 thresholds	$\sim$100K nodes	14,789 weights
Model Size	170 B	8.6 MB	227 KB
CV Accuracy	0.99%	94.13%	**98.41%**
Latency	0.1 ms	13.2 ms	43.7 ms

Person-Aware Cross-Validation: Ensuring Generalization

- **Method:** Group 5-Fold Cross-Validation with person IDs as group identifiers.
- **Guarantee:** No individual appears in both training and test partitions within any fold.
- **Rationale:** Standard random splits risk identity leakage—the model may memorize individual hand morphology rather than learning gesture geometry.
- **Scale:** 3,607 unique subjects ensure diversity in hand shape, skin tone, and gesture style.

The Accuracy-Latency Tradeoff

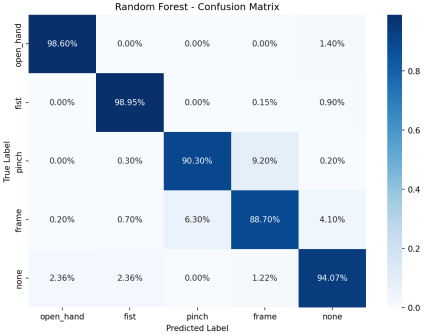
- **Heuristic:** 0.99% at 0.1 ms – fast but non-functional
- **Random Forest:** 94.13% at 13.2 ms – within 60 FPS budget (16.7 ms threshold)
- **MLP:** 98.41% at 43.7 ms – highest accuracy, rate ∼23 FPS
- *Interactive limit:* 23 FPS is sufficient for gesture transitions which occur at human speed (∼1–2 per sec).



Error Distribution: Where Do Models Fail?

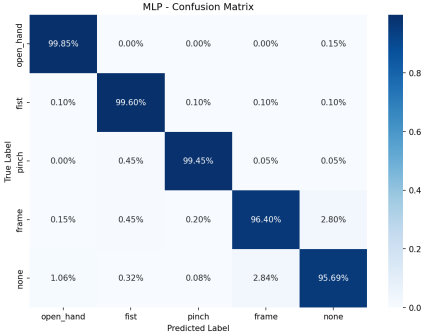
- **Random Forest:** Two-handed "Frame" gesture loses context since joints are processed per hand (F1 = 89.2%).
- **MLP Correction:** Fixes inter-hand context dropping, restoring Frame F1 to 97.3% (an **8.1% bump**).

Random Forest



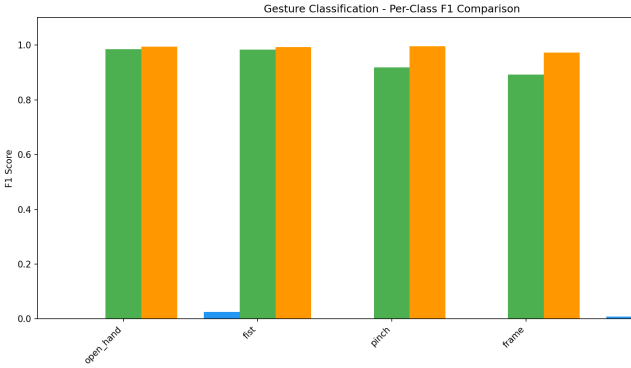
→

MLP



Per-Class F1 Scores: MLP Advantage Across All Categories

Class	RF F1	MLP F1	Delta
open_hand	0.985	0.994	+0.009
fist	0.983	0.993	+0.010
pinch	0.919	0.995	**+0.076**
frame	0.892	0.973	**+0.081**
none	0.919	0.953	+0.034



Generalization Consistency Across Folds

Fold	Heuristic	RF	MLP
1	0.60%	93.45%	98.48%
2	0.98%	94.64%	97.89%
3	1.30%	94.75%	98.54%
4	1.25%	92.09%	98.54%
5	0.81%	95.72%	98.59%

- **MLP:** std $= 0.26\%$, invariant to test population.
- **RF:** std $= 1.25\%$ – fold 4 dip suggests sensitivity to specific subject geometric variations.

System Demonstration: Real-Time Gesture-Controlled Puzzle

- **Platform:** Browser-based application (HTML5 Canvas, JavaScript ES6)
- **Inference stack:** MediaPipe WASM (landmark detection) + ONNX Runtime Web (MLP classification)
- **Architecture:** Fully client-side – no backend server, functions offline after initial load.
- **Temporal smoothing:** 3-frame debounce buffer prevents gesture jitter.

[Live Demonstration in Class]

Implementation Quality: modular Architecture and Testing

- **Test coverage:** 28 automated tests spanning preprocessing, feature extraction, classifiers, and data conversion.
- **Modular design:** Single-responsibility modules with clean interfaces.
- **Reproducibility:** Single command executes the full pipeline from data extraction through evaluation.

Project Directory

```
CVsubject/ml/
  preprocessing.py | features.py | train.py | evaluate.py
  heuristic.py | random_forest.py | mlp.py
  tests/test_core.py # 28 automated tests
```

Current Limitations and Proposed Extensions

Limitations:

- Heuristic fails with 2D landmarks.
- Frame gesture accuracy impaired by per-hand evaluation.
- MLP drops below 60 FPS bound.

Future Directions:

- Model quantization (TFLite, ONNX) for WebAssembly.
- Two-stage frame detector.
- Temporal sequence modeling (LSTM, 1D-CNN).

Summary and Key Findings

- **Data-driven superiority:** ML methods outperform heuristics by two orders of magnitude (98% vs 1%).
- **Highest Accuracy:** MLP achieves 98.41% ($\pm$0.26%) ensuring reliable game interactions.
- **Speed vs. Reliability:** Random Forest meets strict 60 FPS latency constraints but neural networks handle sub-optimal inputs better.
- **Methodology matters:** Person-aware CV prevents reporting inflated accuracies characteristic of identity leakage.

"The gap between classical ML and deep learning is incremental (4 points), but the decision must be governed by practical hardware latency constraints."